Assignment 1 – Weeks 1 and 2

DBMS

Part I:

Q1. DBMS vs. Traditional File System

A traditional file system is essentially just a way to store and organize files on disk — think of it like folders on your computer. Each application manages its own files independently, which sounds fine until things start getting complicated. Redundancy creeps in (the same data stored in multiple places), and there's no built-in way to handle two users editing the same file at the same time, or to recover cleanly if something crashes mid-write.

A Database Management System (DBMS) solves these problems by acting as a smart layer between the application and the actual stored data. It handles concurrent access, enforces data integrity rules, supports structured queries, and offers proper backup and recovery mechanisms.

Concrete scenario: Consider a hospital managing patient records. Multiple doctors and nurses need to read and update the same patient's file simultaneously — one updating medication, another recording vitals. In a file system, these concurrent writes could corrupt the record, or one update could silently overwrite another. A DBMS handles this with transactions and locking, ensuring every update is applied safely and the record stays consistent. It also lets staff query things like "all patients on Drug X" in seconds, which would require scanning through every file manually in a traditional system.

Q2. Weak Entity vs. Strong Entity in ER Diagrams

A strong entity can exist on its own — it has enough attributes to uniquely identify itself without depending on anything else. A weak entity, on the other hand, cannot be uniquely identified by its own attributes alone. It depends on another entity (called its **identifying or owner entity**) for its existence and identification.

In an ER diagram, weak entities are drawn with a double rectangle, and their partial key (called a discriminator) is underlined with a dashed line rather than a solid one.

The primary key of a weak entity is formed by combining the primary key of its identifying entity with its own discriminator attribute. For example, a **Dependent** entity (dependent on an Employee) might have a discriminator like **dependent_name**, and its full primary key would be **(employee_id, dependent_name)**.

The relationship connecting a weak entity to its identifying entity is called an **identifying relationship**, represented in ER diagrams with a double diamond.

Q3. How Should Phone Numbers Be Modeled?

Phone numbers for students should be modeled as a **multivalued attribute**.

Here's the reasoning: a simple attribute holds a single value (like a student's date of birth — there's only one). A composite attribute is made up of multiple components that together form one value (like an address split into street, city, and pin code). A multivalued attribute, however, is one where a single entity instance can have multiple values for the same attribute.

Since a student may have **more than one phone number** (e.g., a personal mobile, a home landline, a work number), phone number fits the definition of a multivalued attribute. In an ER diagram, it would be enclosed in a double ellipse to signal this. In a relational schema, it would typically be broken out into a separate table like **StudentPhone(roll_no, phone_number)** to avoid storing multiple values in a single cell.

Q4. Candidate Keys vs. Super Keys

A **super key** is any set of one or more attributes that uniquely identifies a row in a relation. It can contain extra attributes beyond what's strictly necessary.

A **candidate key** is a **minimal** super key — meaning no attribute can be removed from it without losing the uniqueness property. Every candidate key is a super key, but not every super key is a candidate key.

For the relation Student(roll_no, email, name, branch):

Candidate Keys:

- roll_no — unique per student, and cannot be reduced further
- email — also uniquely identifies a student

Super keys that are NOT candidate keys (have redundant attributes):

- {roll_no, name} — roll_no alone is sufficient, so name is redundant
- {email, branch} — email alone identifies the student; branch adds nothing
- {roll_no, email} — either attribute alone would do the job

Q5. Inserting a Row with a Non-Existent Student ID

Constraint violated: Referential Integrity Constraint (also called a Foreign Key Constraint).

What it protects against: This constraint ensures that relationships between tables remain consistent. Specifically, it prevents the Enrollment table from referencing a student_id that doesn't actually exist in the Student table. Without it, you could end up with "orphan" records — enrollment entries pointing to students who don't exist, which makes the data meaningless and queries unreliable.

What should happen: The insert operation should be rejected outright. The DBMS will throw an error and refuse to commit the insertion. The row will not be added to the Enrollment table until the referenced student_id exists in the Student table.

Q6. Mapping a Many-to-Many (M:N) Relationship

What additional structure must be created:

A new table — commonly called a **junction table** (also known as an associative table or bridge table) — must be created specifically to represent the M:N relationship.

What its primary key consists of:

The primary key of the junction table is typically a **composite key** made up of the foreign keys referencing the primary keys of both participating entities. For example, if Student and Course have an M:N relationship (via Enrollment), the junction table's primary key would be (student_id, course_id).

Why it can't just be a column in one of the existing tables:

In a relational model, a single column holds exactly one value per row. If Student A is enrolled in 5 courses, you cannot store all 5 course IDs in a single column of the Student table without violating first normal form. Similarly, if you added a student_id column to the Course table, you'd only be able to store one student per course, which defeats the whole point of M:N. A junction table sidesteps this by representing each individual (Student, Course) pair as its own row.

Part II:

Part A – Entity and Relationship Identification

Entities and their key attributes:

1. Customer

- customer_id (PK)
- name
- email
- phone_number (multivalued)
- delivery_address

2. Restaurant

- restaurant_id (PK)
- name
- location
- cuisine_type

3. MenuItem

- item_id (PK)
- item_name
- price
- description

4. Order

- order_id (PK)
- order_date
- total_amount
- delivery_status

Relationships and cardinalities:

Customer – Order: 1:N

A customer can place many orders over time, but each individual order belongs to exactly one customer. This is clearly one-to-many.

Order – Restaurant: N:1

Each order is placed at exactly one restaurant, but a restaurant can receive many orders from many customers. This is many-to-one (or equivalently, 1:N from restaurant's side).

Order – MenuItem: M:N

A single order can contain multiple menu items, and the same menu item can appear across many different orders. This is a classic many-to-many relationship.

Restaurant – MenuItem: 1:N

A restaurant offers many menu items, but each menu item belongs to exactly one restaurant (assuming items are restaurant-specific and not shared across restaurants).

Part B – Attribute Classification

One Multivalued Attribute: Customer's phone_number

A customer may have more than one phone number (e.g., a mobile and a home number). Since a single attribute instance can hold multiple distinct values for the same entity, this qualifies as a multivalued attribute. In an ER diagram it would be shown in a double ellipse.

One Composite Attribute: Customer's delivery_address

An address is naturally made up of several sub-parts — house/flat number, street, area, city, state, and pin code. Since it can be meaningfully broken down into components that each carry independent meaning, it qualifies as a composite attribute. Modeling it as composite also makes queries like "find all customers in Kolkata" much simpler.

Part C – Relational Schema for the Junction Table

Since Order and MenuItem have an M:N relationship, we need a junction table. Each row in this table represents one specific menu item within one specific order, along with the quantity ordered.

Schema:

```
OrderItem(order_id, item_id, quantity)
```

Primary Key: (order_id, item_id) — composite primary key. Together they uniquely identify each line item in an order.

Foreign Keys:

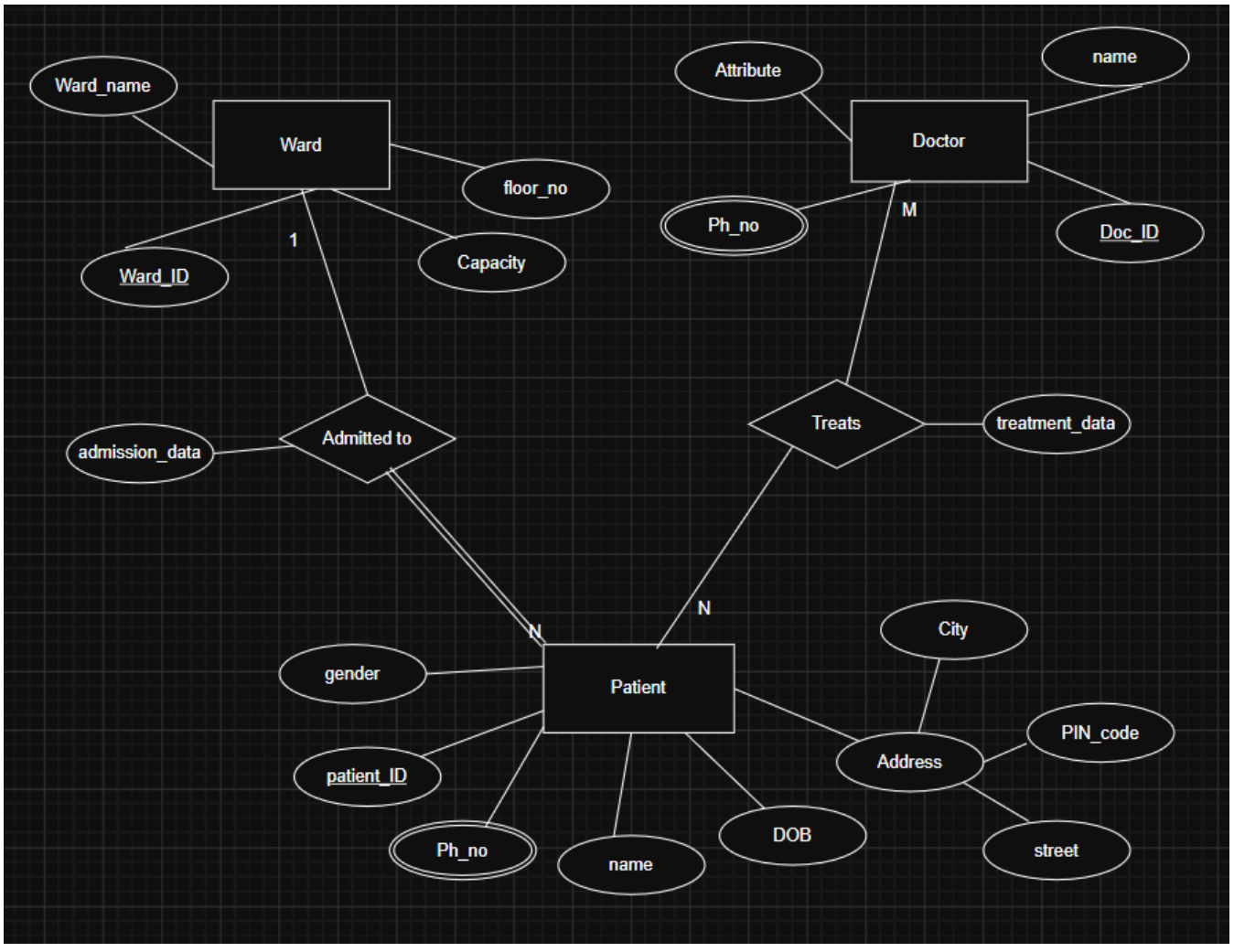
- order_id → Order(order_id)
- item_id → MenuItem(item_id)

Additional attribute:

- quantity (INTEGER, NOT NULL) — records how many units of that menu item were ordered. This is a relationship attribute that only makes sense in the context of a specific order containing a specific item.

Part III: ER Diagram Design

Question 1: HOSPITAL ER DIAGRAM



Question 2: Design Justification – Hospital ER Diagram

Doctor–Patient Relationship (M:N)

The Doctor–Patient relationship is M:N because a single patient can be treated by multiple doctors during their hospital stay (a surgeon, a cardiologist, and a general physician might all be involved), and a single doctor can simultaneously be treating multiple patients across different wards. Neither side is restricted to just one instance of the other.

This M:N relationship gives rise to an associative entity that we might call **Treatment**. Its composite primary key would consist of **(doctor_id, patient_id)**. You could also include **treatment_date** as part of the key if a patient might be treated by the same doctor on multiple separate occasions.

Participation Constraints for Two Relationships:

Ward – Patient (1:N): Total participation from Patient's side

In this hospital scenario, every patient who is admitted must be assigned to a ward — the scenario explicitly states "a patient is admitted to only one ward at a time." The word "admitted" implies that any patient tracked in the system is already in a ward, making this total participation on the patient side. From the ward side, it's partial participation: a ward could theoretically exist in the system (perhaps newly built or temporarily empty) without currently housing any patients.

Doctor – Treatment/Patient (M:N): Partial participation from both sides

Not every doctor in the system is actively treating every patient, and not every patient must have been seen by every doctor. A doctor might be listed in the database but currently have no assigned patients (perhaps they just joined). Similarly, a newly admitted patient might not yet have been assigned to a specific treating doctor. So both sides have partial participation in the M:N treatment relationship.

Attribute Design Decision: Composite Address for Patient

For the Patient entity, I modeled address as a composite attribute broken into street, city, state, and pin_code rather than a single flat string. The reason is practical: hospitals often need to contact or categorize patients by location — for instance, identifying patients from a particular district during a disease outbreak, or routing discharge paperwork by area. Storing address as a single string makes such queries nearly impossible without string parsing, which is fragile. Making it composite allows direct, reliable querying on individual components. Alternatively, phone_number for doctors could have been modeled as multivalued since doctors (like consultants) often have multiple contact numbers, making it appropriate to represent as a double ellipse in the ER diagram.